

Summary

Masked arrays enable the performance and convenience of (rectangular) array computing where missing data would otherwise result in ragged arrays. Some array libraries, such as NumPy [1], PyTorch [2], and Dask [3], offer partial support for masked versions of their arrays/tensors, but these often lack important features, have APIs that are inconsistent with those of the parent namespace, and see only limited support in downstream libraries (e.g. SciPy [4]). Furthermore, there is no standard interface specification, some of these implementations are treated as “experimental” or are maintained less than the parent library, and some automatically mask the result of invalid numerical calculations, hiding bugs and leading to spurious results. MArray [5] offers a new way forward by adding mask support to any array backend that is compatible with the Array API Standard [6].

Advantages over np.nan_

- Works with integer types
- Avoids conflating *missing* and *invalid* data

Advantages over np.ma

- Invalid calculations are *not* masked
- Array API syntax
- Multiple backend support

Acknowledgments

Thanks to everyone who has submitted feature requests, bug reports, and review comments related to these features.

This project has been made possible in part by grant EOSS-0000000432 from the Chan Zuckerberg Initiative DAF, an advised fund of the Silicon Valley Community Foundation.

References

- [1] NumPy, <https://numpy.org/>
- [2] PyTorch, <https://pytorch.org/>
- [3] Dask, <https://www.dask.org/>
- [4] SciPy, <https://scipy.org/>
- [5] MArray, <https://github.com/mdhaber/marray/>
- [6] Array API Standard, <https://data-apis.org/>

Missing data? MArray adds mask support to any array/tensor backend

Getting Started

Any array API compatible backend

New array API compatible namespace with mask support

```
>>> from marray import torch as mxp
>>> x = mxp.asarray([1, 2, 3, 4], mask=[False, True, False, True])
>>> x
MArray(tensor([1, _, 3, _]), tensor([False, True, False, True]))
```

Statistical/Utility Functions

Masked elements are ignored

```
>>> mxp.max(x)
MArray(tensor(3), tensor(False))

>>> mxp.sum(x)
MArray(tensor(4), tensor(False))
```

Sorting Functions

Masked elements are undefined

```
>>> data = [8, 3, 4, 1, 9, 9, 5, 5]
>>> mask = [0, 0, 1, 0, 1, 1, 0, 0]
>>> y = mxp.asarray(data, mask=mask)
>>> print(mxp.sort(y))
[1 3 5 5 8 _ _ _]
```

Elementwise Functions

Mask propagates from input(s) to output

```
>>> print(x**2)
[1 _ 9 _]

>>> print(mxp.sqrt(x))
[1.0000 _ 1.7321 _]
```

Set Functions

Masked values are distinct from unmasked values

```
>>> res = mxp.unique_counts(y)
>>> print(res.values)
[1 3 5 8 _]

>>> print(res.counts)
[1 1 2 1 3]
```

SciPy Compatibility!

Enable support for alternative backends with environment variable SCIPY_ARRAY_API=1

```
>>> from scipy import stats
>>> stats.quantile(x, 1.)
MArray(tensor(3.), tensor(False))
```

MATT HABERLAND^{1,2}, LUCAS COLLEY^{2,3,4}, AND JUSTUS MAGIN^{5,6}

1. CAL POLY, SAN LUIS OBISPO | 2. SCIPY LIBRARY | 3. UNIVERSITY OF OXFORD | 4. PREFIX.DEV | 5. LOPS | XARRAY PROJECT